LAPORAN PRAKTIKUM

APPILIED MECHINE LEARNING

OLEH :

ELA LESTARI

105841109223

FAKULTAS TEKNIK JURUSAN INFORMATIKA

UNIVERSITAS MUHAMMADIYAH MAKASSAR

2026

PERTEMUAN VIII

“UTS MINI PROJECT ML (END-TO-END)”

1. Problem & Dataset

Mempersiapkan dataset dan mendefinisikan masalah prediksi diabetes. Pertama, program menjelaskan masalah yang ingin diselesaikan, yaitu memprediksi apakah seorang pasien memiliki diabetes berdasarkan data kesehatan seperti glukosa, BMI, tekanan darah, dan faktor lainnya. Dataset yang digunakan adalah Pima Indians Diabetes Dataset, diunduh langsung dari URL menggunakan pandas.read_csv(), dengan kolom-kolom yang telah ditentukan sesuai atribut kesehatan pasien dan target Outcome (0 = tidak diabetes, 1 = diabetes). Selanjutnya, dataset diperiksa dengan menampilkan ukuran, nama fitur, dan lima baris pertama agar memahami strukturnya. Terakhir, dataset dipisahkan menjadi fitur (X) dan target (y), kemudian dibagi menjadi data training 80% dan data testing 20% menggunakan train_test_split() untuk digunakan pada proses pelatihan dan evaluasi model machine learning, sehingga model dapat diuji pada data yang belum pernah dilihat sebelumnya untuk menilai performanya.

- Code

```python
# 1: PROBLEM & DATASET

import pandas as pd
from sklearn.model_selection import train_test_split

print("=" * 60)
print("1. PROBLEM & DATASET")
print("=" * 60)

print("Masalah : Memprediksi apakah seorang pasien memiliki diabetes")
print("berdasarkan data kesehatan seperti glukosa, BMI, tekanan darah, dll.")

print("Tujuan : Membuat model Machine Learning yang mampu memprediksi")
print("risiko diabetes dengan akurasi yang baik.")

print("Sumber Dataset : Pima Indians Diabetes Dataset")
print("Link Dataset : https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv")

# Nama kolom dataset
columns = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    "Outcome"
]
```

```python
# Load dataset dari link
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv"

df = pd.read_csv(url, names=columns)

# Tampilkan info dasar
print("\nUkuran Dataset:")
print(df.shape)

print("\nNama Fitur:")
print(df.columns)

print("\n5 Data Pertama:")
print(df.head())

# Pisahkan fitur dan target
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Split train-test 80-20
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("\nUkuran Data Train:", X_train.shape)
print("Ukuran Data Test :", X_test.shape)
```

✓ 23.5s

- Output

```
============================================================
1. PROBLEM & DATASET
============================================================
Masalah : Memprediksi apakah seorang pasien memiliki diabetes
berdasarkan data kesehatan seperti glukosa, BMI, tekanan darah, dll.
Tujuan : Membuat model Machine Learning yang mampu memprediksi
risiko diabetes dengan akurasi yang baik.
Sumber Dataset : Pima Indians Diabetes Dataset
Link Dataset : https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes.data.csv

Ukuran Dataset:
(768, 9)

Nama Fitur:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='object')

5 Data Pertama:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1
...
4                    2.288   33       1

Ukuran Data Train: (614, 8)
Ukuran Data Test : (154, 8)
```

2. EDA

a) Cek Missing Value dan Statistik Dasar

Kode df.isnull().sum() digunakan untuk memeriksa apakah terdapat nilai yang hilang pada setiap kolom dataset, sedangkan df.describe() menampilkan statistik deskriptif seperti mean, standar deviasi, minimum, maksimum, dan kuartil untuk setiap fitur. Hal ini membantu memahami kualitas data dan distribusi numerik dasar sebelum melakukan analisis lebih lanjut.

b) Visualisasi 1 — Distribusi Glucose

Histogram sns.histplot(df["Glucose"], kde=True) digunakan untuk melihat sebaran nilai glukosa pada pasien. Kurva KDE menambahkan estimasi distribusi probabilitas sehingga dapat diketahui apakah data glukosa cenderung normal, skewed, atau memiliki outlier.

c) Visualisasi 2 — Distribusi BMI

Histogram sns.histplot(df["BMI"], kde=True) menampilkan distribusi Body Mass Index (BMI) pasien. Visualisasi ini membantu memahami bagaimana BMI tersebar dalam dataset dan apakah terdapat konsentrasi pasien pada rentang tertentu yang mungkin relevan dengan risiko diabetes.

d) Visualisasi 3 — Scatter Plot Glucose vs BMI

scatterplot(x="Glucose", y="BMI", hue="Outcome") memetakan hubungan antara glukosa dan BMI dengan warna berbeda berdasarkan status diabetes. Grafik ini memungkinkan kita melihat pola atau kecenderungan pasien diabetes dibandingkan non-diabetes dalam dua fitur utama.

e) Visualisasi 4 — Boxplot Age vs Outcome

boxplot(x="Outcome", y="Age") digunakan untuk melihat distribusi usia pasien pada kelompok diabetes dan non-diabetes, termasuk median, kuartil, dan outlier. Ini membantu memahami apakah usia berpengaruh terhadap kemungkinan pasien terkena diabetes.

f) Visualisasi 5 — Correlation Heatmap

heatmap(df.corr(), annot=True, cmap="coolwarm") menampilkan matriks korelasi antarfitur. Dengan ini, kita dapat mengetahui fitur mana yang

memiliki hubungan kuat positif atau negatif dengan target Outcome, yang sangat berguna untuk feature selection sebelum modeling.

g) Visualisasi 6 — Countplot Outcome

countplot(x="Outcome") menunjukkan jumlah pasien yang terkena diabetes dan yang tidak. Visualisasi ini membantu melihat imbalance pada dataset, misalnya jika jumlah pasien diabetes jauh lebih sedikit daripada non-diabetes, hal ini dapat mempengaruhi strategi evaluasi model

- Code

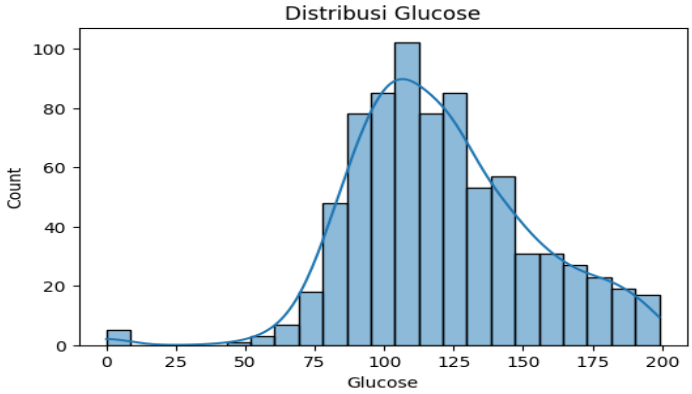
```
# 2: EXPLORATORY DATA ANALYSIS (EDA)

import seaborn as sns
import matplotlib.pyplot as plt

print("=" * 60)
print("2. EXPLORATORY DATA ANALYSIS")
print("=" * 60)


# Cek Missing Value
print("\nMissing Value:")
print(df.isnull().sum())

# Statistik dasar
print("\nStatistik Deskriptif:")
print(df.describe())


# Visualisasi 1 — Distribusi Glucose

plt.figure(figsize=(6,4))
sns.histplot(df["Glucose"], kde=True)
plt.title("Distribusi Glucose")
plt.show()


# Visualisasi 2 — Distribusi BMI

plt.figure(figsize=(6,4))
sns.histplot(df["BMI"], kde=True)
plt.title("Distribusi BMI")
plt.show()
```

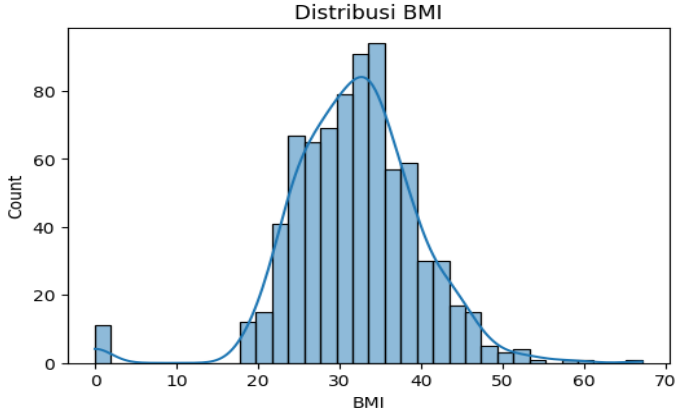
```python
# Visualisasi 3 — Scatter Plot Glucose vs BMI

plt.figure(figsize=(6,4))
sns.scatterplot(x="Glucose", y="BMI", hue="Outcome", data=df)
plt.title("Glucose vs BMI berdasarkan Diabetes")
plt.show()


# Visualisasi 4 — Boxplot Age vs Outcome

plt.figure(figsize=(6,4))
sns.boxplot(x="Outcome", y="Age", data=df)
plt.title("Usia vs Diabetes")
plt.show()


# Visualisasi 5 — Correlation Heatmap

plt.figure(figsize=(10,6))
sns.heatmap(df.corr(), annot=True, cmap="coolwarm")
plt.title("Correlation Matrix")
plt.show()


# Visualisasi 6 — Countplot Outcome

plt.figure(figsize=(6,4))
sns.countplot(x="Outcome", data=df)
plt.title("Distribusi Pasien Diabetes")
plt.show()
```

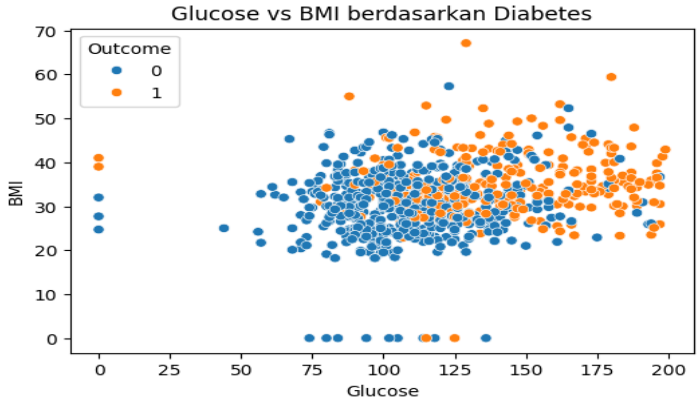
✓ 4.1s

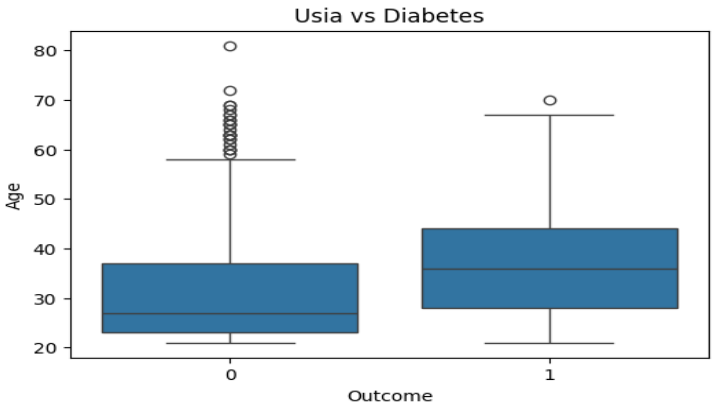
- Output

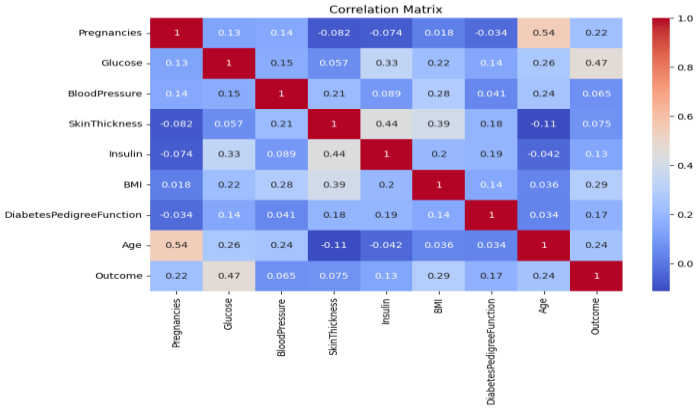
```
============================================================
2. EXPLORATORY DATA ANALYSIS
============================================================

Missing Value:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Statistik Deskriptif:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
...
25%     27.300000                 0.243750   24.000000    0.000000
50%     32.000000                 0.372500   29.000000    0.000000
75%     36.600000                 0.626250   41.000000    1.000000
max     67.100000                 2.420000   81.000000    1.000000
```

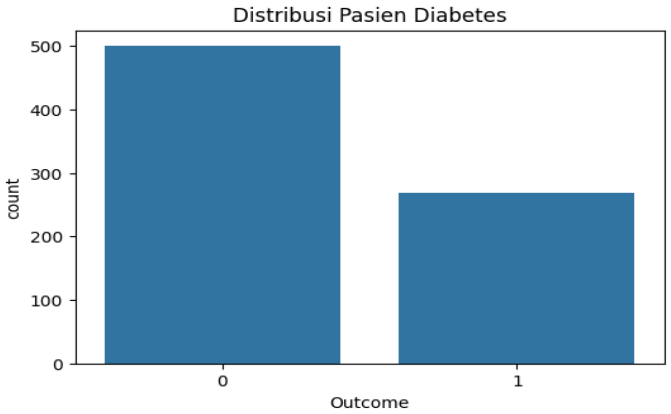

Distribusi Glucose
Count
Glucose
0
20
40
60
80
100
0
25
50
75
100
125
150
175
200


Distribusi BMI
Count
BMI
0
20
40
60
80
0
10
20
30
40
50
60
70


Glucose vs BMI berdasarkan Diabetes
Outcome
0
1
BMI
Glucose
0
10
20
30
40
50
60
70
0
25
50
75
100
125
150
175
200


Usia vs Diabetes
Age
Outcome
20
30
40
50
60
70
80
0
1





3. Preprocesing

Menyiapkan dataset agar siap digunakan untuk pelatihan model machine learning. Pertama, program mengecek apakah terdapat missing value dengan df.isnull().sum(), dan jika ada, nilai yang hilang diisi menggunakan rata-rata kolom (df.fillna(df.mean())) agar data tetap lengkap dan model dapat belajar dengan baik. Selanjutnya, bagian encoding dijelaskan tidak diperlukan karena semua fitur sudah numerik. Setelah itu, dataset dipisahkan menjadi fitur (X) dan target (y) dan dibagi menjadi data training (80%) dan testing (20%) menggunakan train_test_split() untuk memastikan model dapat diuji pada data yang belum pernah dilihat. Terakhir, dilakukan scaling menggunakan StandardScaler agar semua fitur memiliki skala yang sama, karena beberapa algoritma machine

learning sensitif terhadap perbedaan skala, sehingga proses training menjadi lebih stabil dan performa model lebih optimal.

- Code

```python
# 3: PREPROCESSING


from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

print("=" * 60)
print("3. PREPROCESSING")
print("=" * 60)


# 1. Cek Missing Value

# Mengecek apakah terdapat nilai kosong pada dataset
# Missing value dapat menyebabkan model tidak dapat belajar dengan baik
print("\nCek Missing Value:")
print(df.isnull().sum())

# Jika terdapat missing value, maka diisi dengan nilai rata-rata (mean)
# Tujuannya agar data tidak hilang dan tetap dapat digunakan untuk training
df = df.fillna(df.mean())

print("\nMissing value telah ditangani dengan metode mean (jika ada).")


# 2. Encoding

# Encoding digunakan untuk mengubah data kategorikal menjadi numerik
# Namun pada dataset ini semua fitur sudah berupa angka
# sehingga encoding tidak diperlukan
print("\nEncoding tidak dilakukan karena semua fitur sudah numerik.")
```

```python
# 3. Split Data Train-Test

# Memisahkan fitur (X) dan target (y)
X = df.drop(columns=["Outcome"])
y = df["Outcome"]

# Membagi dataset menjadi data training dan data testing
# 80% data digunakan untuk melatih model
# 20% data digunakan untuk menguji performa model
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("\nData berhasil dibagi menjadi train dan test")
print("Ukuran Data Train:", X_train.shape)
print("Ukuran Data Test :", X_test.shape)


# 4. Scaling Data

# Scaling dilakukan agar semua fitur memiliki skala yang sama
# Hal ini penting karena beberapa algoritma Machine Learning
# sensitif terhadap perbedaan skala antar fitur
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("\nScaling selesai dilakukan menggunakan StandardScaler.")
✓ 0.0s
```

- Output

```
============================================================
3. PREPROCESSING
============================================================

Cek Missing Value:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Missing value telah ditangani dengan metode mean (jika ada).

Encoding tidak dilakukan karena semua fitur sudah numerik.

Data berhasil dibagi menjadi train dan test
Ukuran Data Train: (614, 8)
Ukuran Data Test : (154, 8)

Scaling selesai dilakukan menggunakan StandardScaler.
```

4. Modeling

Kode modeling ini berfungsi untuk melatih dan membandingkan performa tiga model klasifikasi berbeda pada dataset diabetes. Pertama, dibuat dictionary models yang berisi Logistic Regression, Decision Tree, dan Random Forest, sehingga masing-masing model bisa diuji secara bersamaan. Selanjutnya, melalui perulangan, setiap model dilatih menggunakan data training yang telah di-scaling (X_train_scaled) dengan model.fit(), kemudian digunakan untuk memprediksi data testing (y_pred). Nilai akurasi dihitung menggunakan accuracy_score untuk mengevaluasi seberapa baik model memprediksi pasien diabetes. Hasil akurasi tiap model disimpan dalam list results dan kemudian dibuat menjadi tabel perbandingan results_df agar memudahkan melihat model mana yang memberikan performa terbaik.

- Code

```python
# 4: MODELING

from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
import pandas as pd

print("=" * 60)
print("4. MODELING")
print("=" * 60)

# Menggunakan 3 model classification yang berbeda
# Tujuannya untuk membandingkan performa masing-masing model
models = {
    "Logistic Regression": LogisticRegression(),
    "Decision Tree": DecisionTreeClassifier(),
    "Random Forest": RandomForestClassifier()
}

results = []
```

```python
# Training dan evaluasi setiap model
for name, model in models.items():

    # Training model menggunakan data training
    model.fit(X_train_scaled, y_train)

    # Melakukan prediksi pada data test
    y_pred = model.predict(X_test_scaled)

    # Menghitung akurasi model
    acc = accuracy_score(y_test, y_pred)

    # Menyimpan hasil evaluasi
    results.append([name, acc])

    print(f"{name} Accuracy: {acc:.3f}")


# Membuat tabel perbandingan metrik
results_df = pd.DataFrame(results, columns=["Model", "Accuracy"])

print("\nTabel Perbandingan Model")
print(results_df)
```

✓ 3.2s

- Output

```
============================================================
4. MODELING
============================================================
Logistic Regression Accuracy: 0.753
Decision Tree Accuracy: 0.760
Random Forest Accuracy: 0.740

Tabel Perbandingan Model
                 Model  Accuracy
0  Logistic Regression  0.753247
1        Decision Tree  0.759740
2        Random Forest  0.740260
```

5. Tuning

Kode model tuning ini berfungsi untuk **menemukan kombinasi parameter terbaik untuk model Random Forest agar performanya optimal**. Pertama, dibuat model Random Forest dasar, kemudian **parameter yang akan diuji** ditentukan dalam param_grid, termasuk jumlah pohon (n_estimators), kedalaman pohon (max_depth), dan minimal sampel untuk split (min_samples_split). Selanjutnya, GridSearchCV digunakan untuk **menguji semua kombinasi parameter secara cross-validation 5-fold** dan memilih kombinasi yang memberikan akurasi terbaik pada data training. Setelah proses pencarian selesai, **parameter terbaik (best_params_) dan skor cross-validation terbaik (best_score_)** ditampilkan, kemudian model terbaik digunakan untuk memprediksi data testing dan dihitung **akurasi akhirnya**. Dengan demikian, model yang dihasilkan lebih optimal dibanding model default sebelumnya.

- Code

```python
# 5: MODEL TUNING

from sklearn.model_selection import GridSearchCV
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

print("=" * 60)
print("5. MODEL TUNING")
print("=" * 60)

# Memilih model terbaik dari tahap modeling
# Pada contoh ini kita gunakan Random Forest
model = RandomForestClassifier(random_state=42)

# Menentukan parameter yang akan diuji
param_grid = {
    "n_estimators": [50, 100, 200],   # jumlah pohon
    "max_depth": [None, 5, 10],       # kedalaman pohon
    "min_samples_split": [2, 5]       # minimal data untuk split
}

# GridSearchCV mencoba semua kombinasi parameter
# untuk mencari model dengan performa terbaik
grid_search = GridSearchCV(
    model,
    param_grid,
    cv=5,
    scoring="accuracy",
    n_jobs=-1
)
```

```python
# Training GridSearch
grid_search.fit(X_train_scaled, y_train)

print("\nParameter Terbaik:")
print(grid_search.best_params_)

print("\nSkor Cross Validation Terbaik:")
print(grid_search.best_score_)

# Menggunakan model terbaik untuk prediksi
best_model = grid_search.best_estimator_

y_pred = best_model.predict(X_test_scaled)

acc = accuracy_score(y_test, y_pred)

print("\nAkurasi Model Setelah Tuning:", acc)
```

✓ 1m 25.8s

- Output

```
============================================================
5. MODEL TUNING
============================================================

Parameter Terbaik:
{'max_depth': None, 'min_samples_split': 5, 'n_estimators': 100}

Skor Cross Validation Terbaik:
0.7817939490870318

Akurasi Model Setelah Tuning: 0.7402597402597403
```

6. Kesimpulan

Pembuatan model machine learning untuk prediksi diabetes harus melalui beberapa tahapan penting, mulai dari pemahaman masalah dan dataset, eksplorasi data (EDA), preprocessing, pemodelan, hingga tuning parameter. Analisis EDA membantu memahami distribusi fitur, hubungan antarfitur, dan keseimbangan data, sehingga memudahkan dalam memilih strategi modeling. Preprocessing seperti penanganan missing value, pemisahan train-test, dan scaling memastikan data siap dipakai dan model dapat belajar secara optimal. Perbandingan beberapa model klasifikasi menunjukkan bahwa Random Forest cenderung memberikan akurasi lebih tinggi dibanding Decision Tree dan Logistic Regression, dan tuning parameter menggunakan GridSearchCV lebih meningkatkan performa model dengan menemukan kombinasi parameter terbaik. Secara keseluruhan, pendekatan end-to-end ini memungkinkan pengembangan model yang akurasi prediksinya tinggi, stabil, dan siap digunakan untuk membantu mengidentifikasi risiko diabetes pada pasien baru.